FlexServ

Documentação técnica do sistema

Gestão de Ordens de Serviço e vendas da TecFlex

Arquitetura, stack, banco de dados, API, front-end, deploy e segurança

Sistema: https://flexserv-eight.vercel.app

Repositório: https://github.com/MuriloMandelli/FlexServ

Outubro de 2026

Sumário

- 1. Visão geral e arquitetura
- 2. Stack tecnológica
- 3. Banco de dados
- 4. Back-end (API)
- 5. Regras de negócio da OS
- 6. Front-end
- 7. Deploy e ambientes
- 8. Segurança
- 9. Como executar o projeto
- 10. Perguntas frequentes

1. Visão geral e arquitetura

O **FlexServ** substitui o controle em papel das Ordens de Serviço (OS) da TecFlex. Antes, o valor era calculado à mão, a letra ficava difícil de ler e encontrar uma OS antiga era demorado. O sistema digitaliza abertura, aprovação eletrônica pelo WhatsApp, acompanhamento, cálculo de valores, vendas de balcão e relatórios.

A arquitetura é **SPA + API REST**: o front-end em React roda no navegador e cuida só da interface; toda regra de negócio e validação fica na API em Node.js/Express, que grava no PostgreSQL.



Front e back são projetos independentes, publicados na Vercel; o front chama /api/... e a Vercel repassa para a API.

> **Resumo para apresentação:** o front mostra e coleta dados; a API valida, aplica as regras e calcula; o banco garante integridade com chaves estrangeiras, campos únicos e transações. Por isso o front nunca envia o valor total da OS: quem calcula é o servidor.

2. Stack tecnológica

Camada	Tecnologia	Motivo da escolha
Linguagem	TypeScript (front e back)	Tipagem detecta erros antes de executar; uma única linguagem para toda a equipe.
Front-end	React 19 + Vite 8	Componentes reutilizáveis; Vite compila e recarrega rápido.
Estilo	Tailwind CSS v4	Estilo direto nas classes, facilita o layout mobile-first.
Navegação	react-router 7	Troca de telas sem recarregar a página (SPA).
Back-end	Node.js + Express 4	Leve e padrão de mercado para APIs REST.
Validação	zod	Valida o JSON recebido antes de chegar ao banco.
Banco	PostgreSQL 18 (Neon)	Relacional, com FKs e transações; gerenciado na nuvem, sem instalação local.
ORM	Drizzle ORM 0.45 + drizzle-kit 0.31	Tabelas definidas em TypeScript e consultas tipadas.
Autenticação	JWT + bcryptjs	Token assinado com validade de 8h; senhas guardadas como hash.
Relatórios	jspdf, jspdf-autotable, recharts	PDF da OS, da venda e das tabelas; gráficos dos relatórios.
Hospedagem	Vercel	Deploy simples, HTTPS automático, API como função serverless.

3. Banco de dados

3.1 Arquivos que ligam o sistema ao banco

A ligação com o banco passa por quatro arquivos, nesta ordem:

Arquivo	Função
`backend/.env`	Guarda a **DATABASE_URL** (endereço e senha do Neon) e o JWT_SECRET. Não vai para o Git; o repositório tem só o modelo `.env.example`. Na Vercel, os valores ficam nas variáveis de ambiente.
`backend/src/db/index.ts`	**Conexão.** Cria o pool de conexões com o Neon e exporta o objeto **db**, importado por todas as rotas que acessam o banco.
`backend/src/db/schema.ts`	**Estrutura das tabelas** em TypeScript: colunas, tipos, chaves estrangeiras, campos únicos e enums.
`backend/drizzle.config.ts`	Configuração do drizzle-kit. O comando `npm run db:push` compara o schema com o banco e cria ou altera as tabelas.

Conteúdo da conexão (*backend/src/db/index.ts*):

```
import { Pool, neonConfig } from "@neondatabase/serverless";
import { drizzle } from "drizzle-orm/neon-serverless";
import ws from "ws";
import * as schema from "./schema";

neonConfig.webSocketConstructor = ws;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
export const db = drizzle(pool, { schema });
```

Exemplo de uso em uma rota: `await db.select().from(cliente).where(eq(cliente.id, id))`. O Drizzle traduz para SQL e o TypeScript confere se a tabela e as colunas existem.

3.2 Tabelas e relacionamentos

Tabela	Conteúdo	Relacionamentos
loja	Lojas da empresa	1:N vendedor, OS e venda
vendedor	Usuários; papel vendedor ou gestor; senha em hash	N:1 loja
cliente	Nome, CPF/CNPJ (único), contato, endereço	1:N equipamento e OS
equipamento	Máquinas do cliente: tipo, marca, modelo, nº de série	N:1 cliente; 1:N OS
item	Catálogo: peça, máquina ou mão de obra, com preço e custo	Usado em servico_item e venda_item
status_os	Status configuráveis (nome, cor, ordem) ligados a uma fase	1:N OS
servico	A OS: oficina/campo, defeito, diagnóstico, garantia, valores, pagamento, aceites eletrônicos	N:1 cliente, status, loja; quem abriu e técnico (vendedor); máquina opcional
servico_item	Itens da OS com quantidade e preço congelado	Tabela associativa N:N (servico × item)
venda	Venda de balcão: valores, pagamento, situação	N:1 loja, vendedor, cliente (opcional)
venda_item	Itens da venda com preço congelado	N:N (venda × item)
aviso_cliente	Registro do "Já avisei" pelo WhatsApp	N:1 OS ou venda
configuracao	Configurações editáveis pelo gestor (termos de aprovação)	Chave e valor

3.3 Decisões de modelagem

- **Preço congelado:** ao incluir um item na OS ou na venda, o preço e o custo são copiados para a tabela de itens. Mudar o catálogo depois não altera documentos antigos (mesma lógica de uma nota fiscal).
- **Total calculado pelo banco:** a função `recalcularTotal` (backend/src/services/os.ts) soma quantidade × preço no próprio PostgreSQL, com tipo numeric. Evita o erro de ponto flutuante do JavaScript e impede que o valor seja adulterado pelo navegador.
- **Status livres, regras fixas:** a empresa cria e renomeia status à vontade, mas cada um pertence a uma das 5 fases (aberta, execução, concluída, entregue, cancelada). As regras de negócio olham a fase, nunca o nome.
- **Transações:** abrir uma OS cria a OS, insere os itens e calcula o total dentro de `db.transaction`. Se qualquer etapa falhar, nada é gravado (rollback).
- **Integridade:** chaves estrangeiras impedem registros órfãos; CPF/CNPJ, e-mail de usuário e nome de status são únicos. Erros do Postgres (23505 duplicado, 23503 em uso) viram mensagens amigáveis em `utils/http.ts`.
- **Nomenclatura:** snake_case no banco (`valor_total`) e camelCase no TypeScript (`valorTotal`), com o mapeamento feito pelo Drizzle.

4. Back-end (API)

4.1 Estrutura de pastas

```
backend/src/
├── index.ts            Ponto de entrada: cria o Express e registra as rotas
├── db/                 index.ts (conexão), schema.ts, seed.ts, seed-demo.ts
├── middleware/auth.ts  autenticar (confere o JWT) e apenasGestor (permissão)
├── routes/             auth, clientes, lojas, vendedores, itens, equipamentos,
│                       status, os, vendas, relatorios
├── services/os.ts      Regras de negócio da OS e cálculo do total
└── utils/              validacao.ts (schemas zod) e http.ts (tratamento de erros)
```

4.2 Caminho de uma requisição

Exemplo: mudar o status da OS 12 (`PATCH /os/12/status`).

- **index.ts** recebe a requisição; `express.json()` converte o corpo em objeto.
- **autenticar** (middleware/auth.ts) exige o header `Authorization: Bearer <token>`. Token ausente, alterado ou vencido → **401**. Válido → os dados do usuário ficam em `req.usuario`.
- **A rota** (routes/os.ts) valida o corpo com zod.
- **A regra** `validarTrocaDeFase` (services/os.ts) verifica se a mudança é permitida.
- Regra violada → **400** `{ "erro": "mensagem" }`. Permitida → grava e responde **200** com a OS atualizada.

4.3 Códigos HTTP usados

Código	Significado no FlexServ
200 / 201	Sucesso / registro criado
204	Sucesso sem conteúdo (exclusão)
400	Dado inválido ou regra de negócio violada
401	Não autenticado (sem token ou token vencido)
403	Sem permissão (ação exclusiva do gestor)
404	Registro não encontrado
409	Conflito: duplicado ou registro em uso
500	Erro interno inesperado

4.4 Principais rotas

Rota	Acesso	Descrição
POST /login	público	Recebe e-mail e senha, devolve o token JWT
GET /health	público	Confirma que a API e o banco estão no ar
/clientes, /clientes/:id/equipamentos	logado	Clientes e suas máquinas
/itens, /lojas	leitura logado, escrita gestor	Catálogo e lojas
/vendedores, /status	gestor (status: leitura logado)	Usuários e status configuráveis
GET/POST /os, GET/PUT /os/:id	logado	Lista com filtros, abertura com itens, detalhe e edição
POST /os/:id/aprovacao	logado	Gera os links de aprovação e congela os termos
POST /os/:id/aprovacao/tecnico	técnico da OS	Técnico assina direto no sistema
GET/POST /publico/aprovacao/:token	público (token)	Página de aprovação aberta pelo WhatsApp
/configuracoes/termos	leitura logado, escrita gestor	Texto dos termos de aprovação
PATCH /os/:id/status	logado	Troca de status com validação por fase
PUT /os/:id/pagamento	logado	Desconto, forma e situação de pagamento
/vendas	logado	Vendas de balcão: criação, pagamento, cancelamento
GET /relatorios?de&ate	logado	Faturamento, lucro, OS por status, top itens, por vendedor

5. Regras de negócio da OS

Ficam centralizadas em *backend/src/services/os.ts*, para valerem em qualquer rota:

- A OS só sai da fase **aberta** depois de **aprovada** pelo cliente e pelo técnico responsável (assinatura eletrônica).
- O técnico responsável pode ser diferente de quem abriu a OS; a máquina é opcional (o serviço costuma envolver várias).
- Se itens ou desconto mudarem antes de iniciar, a aprovação cai e é preciso assinar de novo: o cliente nunca aprova um valor e paga outro.
- Não é possível **concluir** sem itens nem **entregar** sem concluir.
- **Entregue** e **cancelada** são fases finais: a OS vira histórico.
- Itens só podem ser alterados nas fases aberta e em execução.
- Atendimento **em campo** exige endereço; guarda a data da visita.
- Desconto nunca maior que o valor dos itens; total nunca negativo.
- Vendas de balcão não aceitam mão de obra (serviço sempre vai em OS).

6. Front-end

```
frontend/src/
├── main.tsx / App.tsx  Inicia o React e define as rotas (telas)
├── api.ts              Toda chamada à API passa por aqui
├── auth.tsx            Usuário logado e proteção das telas
├── pages/              Uma tela por arquivo (NovaOS, DetalheOS, Vendas, Relatorios...)
├── components/         Peças reutilizáveis (Tabela, AprovacaoOS, WhatsApp, Layout...)
├── pdf.ts              PDF da OS, da venda e das tabelas
└── formatar.ts         Moeda, datas e cores dos status
```

- **api.ts** é a ponte com o back-end: adiciona o token, converte JSON, transforma erros em mensagens e, ao receber 401, encerra a sessão.
- **auth.tsx** guarda o token no navegador; o componente `RotaProtegida` redireciona para o login quem não está autenticado.
- **Proxy /api:** o front sempre chama `/api/...`. Em desenvolvimento o Vite repassa para `localhost:3333` (vite.config.ts); em produção a Vercel repassa para a API (frontend/vercel.json). Como tudo parece ser o mesmo endereço, não é preciso configurar CORS.
- **Mobile-first:** barra de navegação inferior no celular e menu lateral no computador, pois a empresa usa muito o celular.
- **Aprovação eletrônica:** "Enviar para aprovação" gera um link para o cliente e outro para o técnico, enviados pelo WhatsApp. A página pública (`/aprovar/:token`) mostra o orçamento e os termos; a pessoa marca "Li e concordo" e toca em "Assinar". O técnico logado pode assinar na própria tela da OS. A tela da OS se atualiza sozinha quando alguém assina.
- **Fluxo guiado:** linha do tempo (Aberta → Aprovada → Em execução → Concluída → Entregue) e um botão com o próximo passo; aviso ao sair com alterações não salvas.
- **WhatsApp:** link `wa.me` com mensagem pronta (sem API paga); o botão "Já avisei" registra o aviso no banco.
- **Tabelas:** ordenação clicando no cabeçalho, exportação em PDF e exibição em cartões no celular.

7. Deploy e ambientes

Ambiente	Front-end	API	Banco
Local	http://localhost:5173	http://localhost:3333	Neon (mesmo banco)
Produção	https://flexserv-eight.vercel.app	https://flexserv-api.vercel.app	Neon (mesmo banco)

- Dois projetos separados na Vercel. A API roda como **função serverless**: o `index.ts` exporta o app e só abre a porta quando não está na Vercel.
- `backend/vercel.json` indica o framework Express; `frontend/vercel.json` indica Vite, repassa `/api` para a API e faz o fallback da SPA (qualquer rota abre o index.html).
- Segredos (DATABASE_URL e JWT_SECRET) ficam nas variáveis de ambiente da Vercel. Os arquivos `.vercelignore` impedem o envio do .env.
- Para publicar uma alteração: `vercel --prod` dentro de backend/ ou frontend/.

8. Segurança

- **Senhas** com hash bcrypt; o hash nunca é devolvido pela API.

- **JWT** com id, e-mail, papel e loja, assinado com JWT_SECRET e válido por 8 horas. Qualquer alteração no token invalida a assinatura.
- **Permissões:** o middleware `apenasGestor` protege usuários, lojas, catálogo e status. O gestor não pode excluir nem rebaixar a própria conta.
- **Validação no servidor:** o front valida para ajudar o usuário, mas quem garante é a API (zod + regras de negócio).
- **Segredos fora do Git:** .env no .gitignore; o repositório tem apenas o .env.example.
- **Links de aprovação:** token aleatório de 48 caracteres, válido só enquanto a OS aguarda aprovação; a página pública não mostra CPF, telefone nem endereço.
- **Comprovante do aceite:** nome, data/hora, IP e aparelho, com cópia dos termos aceitos congelada na OS.
- **HTTPS** automático em produção pela Vercel.

9. Como executar o projeto

Pré-requisito: Node.js instalado. São dois terminais, um para cada parte.

```
# 1. API
cd backend
npm install
copy .env.example .env      (preencher DATABASE_URL e JWT_SECRET)
npm run db:push             (cria as tabelas, só na primeira vez)
npm run db:seed             (loja e usuário iniciais)
npm run db:seed-demo        (opcional: dados de demonstração)
npm run dev                 -> http://localhost:3333

# 2. Front-end
cd frontend
npm install
npm run dev                 -> http://localhost:5173
```

Para testar no celular, use o endereço "Network" exibido pelo Vite, com o celular na mesma rede Wi-Fi.

10. Perguntas frequentes

Onde o sistema se conecta ao banco?

Em backend/src/db/index.ts, usando a DATABASE_URL do .env. As tabelas estão em schema.ts e todas as rotas importam o objeto db.

Por que usar ORM e não SQL puro?

Pela tipagem: um nome de coluna errado é apontado pelo TypeScript antes de executar. Onde é necessário, como no cálculo do total, usamos SQL direto pelo recurso sql do Drizzle.

Como garantir que o valor da OS está certo?

O banco calcula o total a partir dos itens com preço congelado. O front-end apenas exibe.

E se uma etapa falhar ao salvar a OS?

Tudo acontece em uma transação; em caso de erro, nada é gravado.

Por que front e back separados?

Separação de responsabilidades: a API pode atender outro front (um app, por exemplo) sem mudanças, e cada parte é publicada de forma independente.

Por que a OS precisa de aprovação?

Para registrar o aceite do cliente sobre o valor. Mudou item ou desconto antes de iniciar, a aprovação é refeita.

A assinatura pelo link tem validade?

É uma assinatura eletrônica simples: registra quem aceitou, quando, de qual IP e aparelho, junto com o texto exato dos termos aceitos.

Como os status podem ser livres sem quebrar as regras?

Cada status pertence a uma fase fixa e as regras olham a fase. A empresa personaliza os nomes; o comportamento continua garantido.